1️⃣ What is an Interface in C#?

An **Interface** in C# is a **contract** that defines a set of methods, properties, events, or indexers that a class must implement.

It specifies **WHAT a class should do**, but **not HOW it does it**.

Think of an interface like a **blueprint or agreement**.

Example Analogy

Imagine a **Remote Control Interface**.

The interface defines buttons:

- PowerOn()
- PowerOff()
- VolumeUp()

But the **TV manufacturer decides how these buttons work internally**.

2️⃣ Syntax of an Interface

```
interface IShape
{
    double CalculateArea();
}
```

Important points:

✓ Uses the keyword **interface**
✓ By convention, interface names start with **I**
✓ Contains **method declarations only (no implementation)**

3️⃣ Implementing an Interface

A class implements an interface using the **: symbol**

Example

```
using System;

interface IShape
{
    double CalculateArea();
}

class Circle : IShape
{
    public double Radius { get; set; }

    public double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

class Program
{
    static void Main()
    {
        Circle circle = new Circle();
        circle.Radius = 5;

        Console.WriteLine(circle.CalculateArea());
```

```
    }
}
```

Output

78.5398

Here:

- IShape defines the contract
- Circle implements the interface

4️⃣ Why Do We Use Interfaces?

Interfaces are used for **loose coupling and abstraction**.

Benefits:

✓ Code flexibility
✓ Easier unit testing
✓ Dependency Injection support
✓ Multiple inheritance support

Example in **ASP.NET Core**

```
public interface IUserService
{
    User GetUser(int id);
}
```

Implementation

```
public class UserService : IUserService
{
    public User GetUser(int id)
    {
        return new User { Id = id, Name = "John" };
```

```
    }
}
```

Controller uses interface

```
public class UserController
{
    private readonly IUserService _service;

    public UserController(IUserService service)
    {
        _service = service;
    }
}
```

This enables **Dependency Injection**.

---

5 Rules of Interfaces

Interfaces follow some rules:

Rule	Description
No method body	Only method signatures
No fields	Only properties
Methods are public	By default
Cannot create object	Must implement in class
Supports multiple inheritance	A class can implement many interfaces

---

6 Interface with Multiple Methods

```
interface ICalculator
{
    int Add(int a, int b);
    int Subtract(int a, int b);
}
```

Implementation

```
class Calculator : ICalculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Subtract(int a, int b)
    {
        return a - b;
    }
}
```

7️⃣ Interface with Properties

Interfaces can contain **properties**.

```
interface IEmployee
{
    int Id { get; set; }
    string Name { get; set; }
}
```

Implementation

```
class Employee : IEmployee
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

8️⃣ Interface with Multiple Inheritance

C# **does not support multiple inheritance with classes**, but **interfaces allow it**.

Example

```
interface IPrinter
{
    void Print();
}

interface IScanner
{
    void Scan();
}

class MultiFunctionPrinter : IPrinter, IScanner
{
    public void Print()
    {
        Console.WriteLine("Printing...");
    }
```

```
    public void Scan()
    {
        Console.WriteLine("Scanning...");
    }
}
```

This is **very common in enterprise applications**.

9️⃣ Explicit Interface Implementation

Sometimes two interfaces have **same method names**.

```
interface IFirst
{
    void Show();
}

interface ISecond
{
    void Show();
}
```

Implementation:

```
class Demo : IFirst, ISecond
{
    void IFirst.Show()
    {
        Console.WriteLine("IFirst Show");
    }
```

```
    void ISecond.Show()
    {
        Console.WriteLine("ISecond Show");
    }
}
```

Usage

```
Demo d = new Demo();

((IFirst)d).Show();
((ISecond)d).Show();
```

Output

```
IFirst Show
ISecond Show
```

🔟 Default Interface Methods (Modern C#)

Starting **C# 8**, interfaces can contain **method implementations**.

Example

```
interface ILogger
{
    void Log(string message)
    {
        Console.WriteLine(message);
    }
}
```

Class implementing it:

```
class FileLogger : ILogger
{
}
```

Usage:

```
FileLogger logger = new FileLogger();
logger.Log("Application Started");
```

1️⃣2️⃣ Interface vs Abstract Class

Feature	Interface	Abstract Class
Implementation	Mostly no	Can contain
Fields	Not allowed	Allowed
Multiple inheritance	Yes	No
Constructor	No	Yes
Use case	Contract	Base class

1️⃣3️⃣ Common Interfaces in .NET

Some built-in interfaces you already use:

Interface	Purpose
IEnumerable	Iteration
IDisposable	Resource cleanup
IComparable	Object comparison
ICloneable	Cloning objects

Interface	Purpose
ILogger	Logging